Data Models

1. Data Model:
   - A set of concepts to describe:
     a. Structure of database
        - Defines how data is stored & organized, stored, and related within the system
        - Key components include:
          - Relations b/w tables
          - Attributes (columns)
          - Records (rows)
     b. Operations for manipulating these structures
     c. Certain constraints that the database should obey
   - Constructs are used to define the database structure
     - A table
     - Attributes in that table
     - Their relationships to other data.
   - Operations can be built in or user defined.

Categories of Datamodel:

1. Conceptual (high level, semantic) data models:
   - The way users percieve data
2. Physical (low level, internal) data models:
   - The way data is actually stored (Trees, hashmaps)
3. Implementation (representational) data models:
   - Links the above two
   - Translates conceptual into physical automatically
   - Used by many commercial DBMS

Definitions:

Database Schema. ↗ intension
- Desc of a database
- Changes very infrequently

Schema Diagram:
↳ An illustrative diagram of a database schema

Schema Construct:
↳ Desc of a particular table or attribute or other objects in the database.

Database state: → extension
↳ Actual data stored in a database at a particular moment in time.
↳ Also called database instance or occurence or snapshot
↳ Also applied to individual components
↳ Changes every time database is updated

Intial Data base state:
↳ Database state when it is first loaded into the system

Valid State:
↳ A state that satisfies the structure and constraints of the database.

Example,

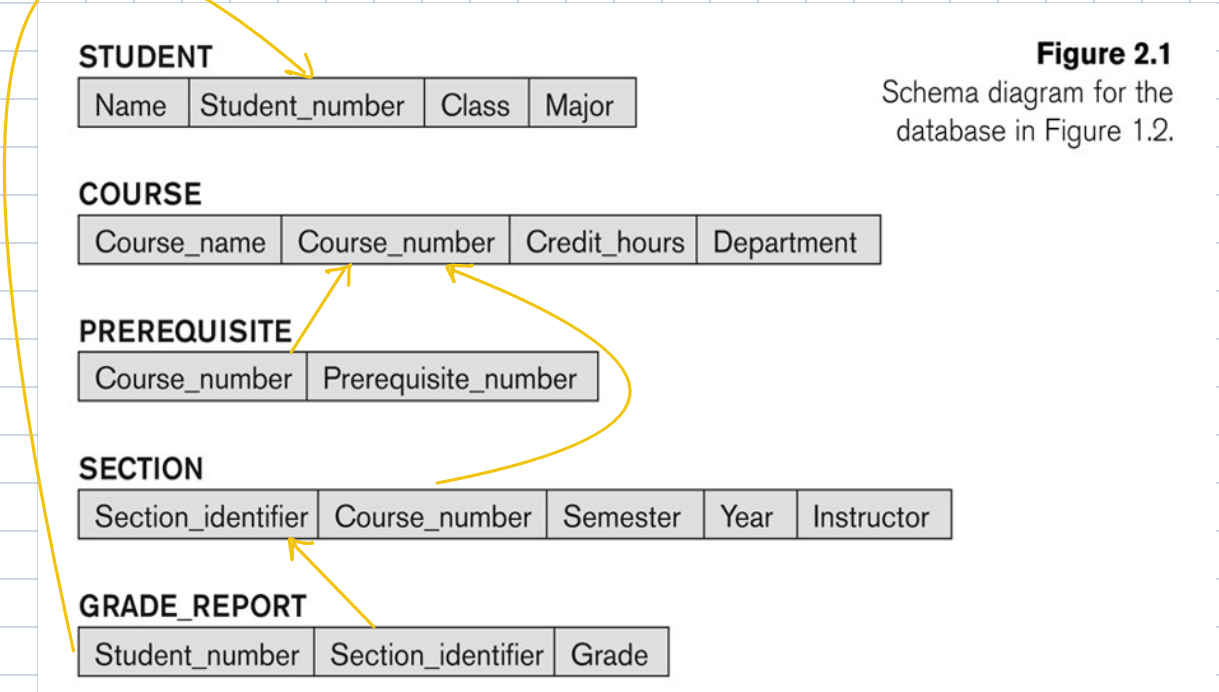


Figure 2.1
Schema diagram for the database in Figure 1.2.

Foreign key
↓
Primary key

Three Schema Architecture

↳ Proposed to support:
1. Program – data independence
2. Support of multiple views of data.

↳ Following are the three levels:

1. Internal Schema:
   ↳ Desc physical storage structures
   ↳ Uses physical data model

2. Conceptual Schema:
   ↳ Desc structure & constraints for the whole database.
   ↳ Uses conceptual or implementation data model

3. External Schema:
   ↳ Desc various views for different users

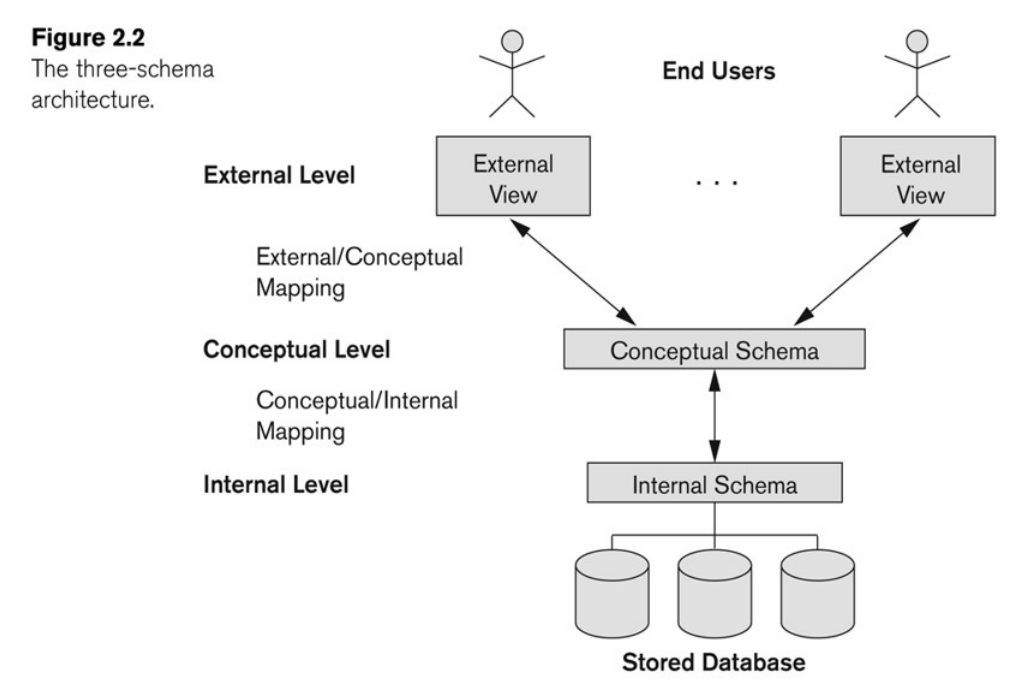


Figure 2.2
The three-schema architecture.

- Mappings among schema levels are needed to transform requests and data.
- Programs refer to an external schema, and are mapped by the DBMS to the internal schema for execution.
- Data extracted from the internal DBMS level is reformatted to match the user's external view (e.g. formatting the results of an SQL query for display in a Web page)

↳ Data independence

1. Logical data independence → changing conceptual has no effect on external
2. Physical data independence
   ↳ Changing internal has no effect on conceptual

DBMS Languages:

1. Data Definition Language (DDL):

↳ To specify the conceptual schema
↳ Used to define internal and external schema
↳ In some DDL, storage Definition Language (SDL) and View definition Language (VDL) are used to define internal and external schemas respectively.

2. Data Manipulation Language (DML):

↳ To specify database retrievals and updates
↳ Can be embedded in C, C++, python etc.
↳ Types of DML:

a. High Level or Non procedural language:

↳ You don't define the procedure to access data

b. Low Level or procedural Language:

↳ You have to describe the procedure to access data.

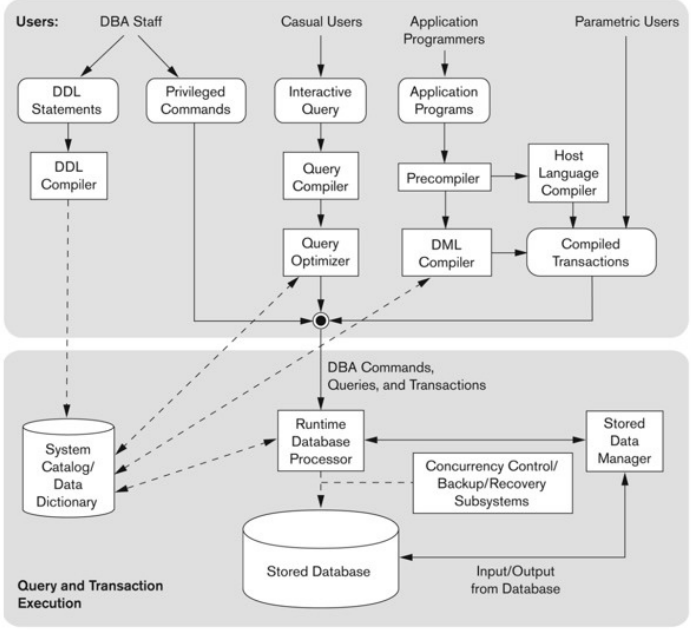
Example,

```
for i in range(n):
    a[i].print()
```

↑ array of obj of a class

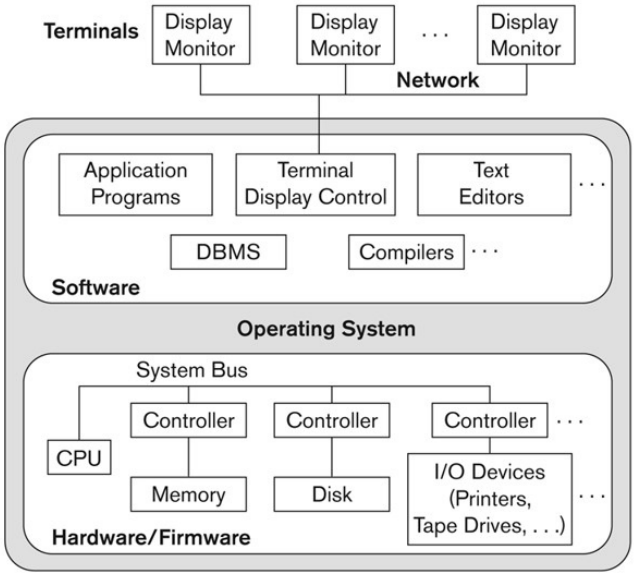
Database System Utilities:

- **Data Loading:** Tools for loading data from files into the database, including conversion tools.
- **Database Backup:** Periodic backups of the database, typically onto tape.
- **Reorganization:** Tools for reorganizing database file structures for efficiency.
- **Report Generation:** Utilities for generating reports from the database.
- **Performance Monitoring:** Tools for monitoring and optimizing database performance.
- **Other Functions:** Includes tasks like sorting, user monitoring, and data compression.



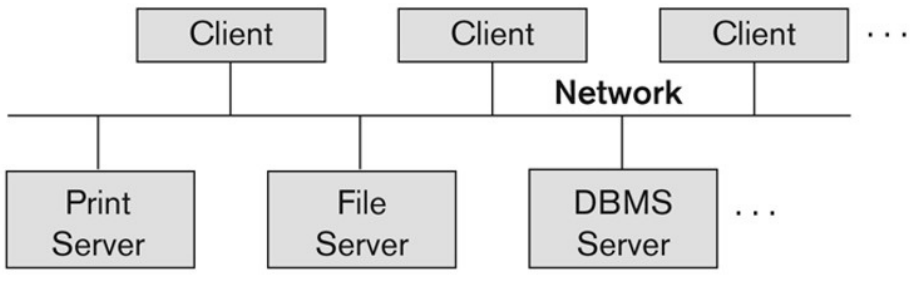
Centralized DBMS Architecture:

↳ Combines everything into a single system DBMS software, hardware, application programs, and user interface processing.



The difference between 2-tier and 3-tier client-server architectures lies in the number of layers between the client and the data, and the responsibilities of each layer. Let's break them down:

- **2-Tier Client-Server Architecture:**
  - In a 2-tier architecture, there are two layers:
    - **Client Layer (also called the Presentation Layer):** This is where the client (the user interface or application) interacts with the server. The client requests data and services from the server.
    - **Server Layer (also called the Database Layer):** This layer is typically where the database or data source is hosted. It directly handles the requests from the client and provides data or performs actions (like CRUD operations — Create, Read, Update, Delete).
  - **How it works:**
  - The client communicates directly with the server to request and manipulate data.
  - The client may include the user interface and business logic. However, much of the work (especially regarding data management) is performed by the server.
  - **Example:**
  - A desktop application (client) sends a request to a database server (e.g., MySQL or PostgreSQL) to fetch or update data.
  - The client handles both the user interface and logic,

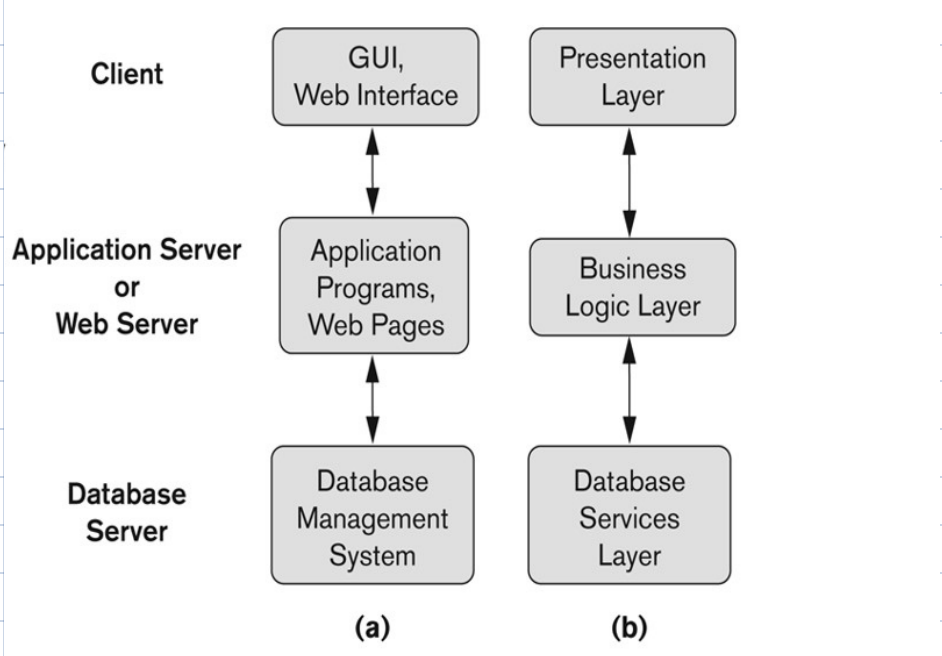


while the server manages and stores the data.

- **3-Tier Client-Server Architecture:**
  - In a 3-tier architecture, there are three layers:
    - **Client Layer (Presentation Layer):** This layer contains the user interface. The client communicates with the application server (middle tier) to request data or services.
    - **Application Layer (Business Logic Layer):** This is the middle layer. It contains the business logic and processes the requests from the client before sending them to the database. It can also handle authentication, authorization, data validation, and other services. It's responsible for processing the logic of an application.
    - **Database Layer (Data Layer):** This is where the actual data is stored. The application server communicates with the database server to retrieve or manipulate data.

  - **How it works:**
  - The client sends requests to the application server (middle tier).
  - The application server handles the logic and processes the request (e.g., checks for business rules, validation, etc.).
  - The application server communicates with the database server to retrieve or modify data.
  - Once the data is retrieved or updated, it is sent back to the client through the application server.

  - **Example:**
  - A web browser (client) sends a request to an application server (middle tier) for user data.
  - The application server processes the request, communicates with the database server to fetch the data, and then sends it back to the client.



Distributed DBMS types:

1. Homogeneous DDBMS:
   - Same Data model schema structure across the distributed

severs.

2. Heterogeneous DDBMS:
   ↳ Different Data model schema structure for every distributed severs.

↳ Federated or Multidatabase system provides a global view of the data in heterogenous DDBMS.

History of Data Models:

- **Network Model:**
  - The Network Model organizes data in a graph structure, where entities (records) are nodes and relationships between them are edges (links).
  - It supports many-to-many relationships, allowing more complex connections than the hierarchical model.
  - Data can be navigated by following links between records.
  - **Example:**
    - CODASYL (Conference on Data Systems Languages).

- **Hierarchical Model:**
  - The Hierarchical Model organizes data in a tree-like structure, with records represented as nodes and each record having a single parent (except the root).
  - It supports one-to-many relationships, where a parent can have multiple child records, but each child has only one parent.
  - It's efficient for hierarchical data but lacks flexibility for other relationships.
  - **Example:**
    - IMS (Information Management System) by IBM.

- **Relational Model:**
  - The Relational Model represents data in tables (relations), with rows (tuples) and columns (attributes). It is based on set theory and uses SQL for querying.
  - Relationships between tables are defined with keys (primary and foreign). Data is logically organized.
  - **Example:**
    - MySQL, PostgreSQL, Oracle.

- **Object-Oriented Data Models:**
  - The Object-Oriented Model stores data as objects, similar to object-oriented programming languages.
  - Each object represents both data (attributes) and functions (methods) that operate on the data. It allows for inheritance, encapsulation, and polymorphism, making it suitable for modeling complex entities.
  - **Example:**

        - GemStone/S, ObjectDB.

- **Object-Relational Models:**
    - The Object-Relational Model combines features of both the relational and object-oriented models. It allows relational databases to store complex data types (like objects and arrays) while maintaining the simplicity of relational tables.
    - It supports object-oriented features like inheritance and integrates them with relational data storage and querying.
    - **Example:**
        - PostgreSQL supports object-relational features with user-defined types and functions.